

Guia Prático: Comandos Úteis de Git e Github



Por que você não será contratado sem isso?

Vou te mandar uma real logo de cara: você pode passar meses estudando Python ou qualquer outra linguagem de programação, decorar todas as funções do mundo e saber criar lógicas incríveis. Mas, se você chegar em uma entrevista de emprego ou tentar um freela e não souber usar o **Git** e o **GitHub**, você está fora do jogo.

O mercado de tecnologia não quer apenas quem sabe escrever código. Ele quer quem sabe trabalhar como um profissional. E, no dia a dia de uma empresa, ninguém trabalha sozinho e ninguém envia código por anexo no WhatsApp.

Tudo passa pelo Git (o sistema que controla as versões do código e impede que você apague seu projeto por erro) e pelo GitHub (a rede social onde os programadores expõem seus trabalhos e os recrutadores caçam talentos).

O Erro de 90% dos Tutoriais

Se você já tentou aprender Git antes e achou confuso, cheio de termos chatos como *"staging area"*, *"repositório remoto"* ou *"árvore de diretórios"*, a culpa não é sua. A maioria dos materiais foca na teoria e tenta te ensinar usando projetos gigantescos.

Aqui nós vamos fazer o oposto. Este material é um **Laboratório Prático**.

Nas próximas páginas, nós vamos criar um arquivo de texto ridiculamente simples e vamos usar o Git para entender, na prática, como criar um cofre de segurança para o seu código e como viajar no tempo para desfazer erros graves.

Você não vai apenas ler. Você vai abrir o seu editor de código, digitar os comandos e ver a mágica acontecer na sua tela. No final, você vai dar o seu primeiro `git push` e colocar seu primeiro projeto no ar.

Sem enrolação, sem teoria chata.

Abraços,

Wollacy Willyan de Carvalho

[Instagram](#)

[Devzinho](#)

[LinkedIn](#)

[E-mail](#)

Pegue o seu café, abra o seu terminal e vamos para a prática.

🔗 Índice: Laboratório Git

Capítulo 1: O "Hello World" (Seu Primeiro Repositório)

- Criamos uma pasta no computador e um arquivo `index.html` apenas com um `<h1>Olá, Mundo!</h1>`.
- Aqui a gente ensina:
 - `git init` (Transformando a pasta normal em um cofre do Git).
 - `git add index.html` (Colocando o arquivo na área de preparação).
 - `git commit -m "Meu primeiro commit"` (Batendo a foto oficial da versão 1).

Capítulo 2: A Evolução (Criando Histórico)

- Pedimos para o aluno adicionar um parágrafo no HTML: `<p>Estou aprendendo Git!</p>`.
- Ensinamos o `git status` para ele ver que o Git percebeu a alteração.
- Repetimos o ciclo: `add` e `commit -m "Adicionando parágrafo"`.
- Agora vem o pulo do gato: ensinamos o `git log` para ele ver a linha do tempo com as duas versões salvas.

Capítulo 3: A Máquina do Tempo (Botão de Pânico)

- Pedimos para ele apagar o HTML inteiro e salvar (simulando aquele erro clássico de iniciante que dá desespero).
- Ensinamos o comando mágico (`git restore index.html` ou `git checkout`) e... BOOM! O HTML volta intacto.
- *Efeito Uau:* É aqui que o aluno pensa: "Caramba, o Git salvou a minha vida!".

Capítulo 4: Subindo para as Nuvens (O GitHub)

- Agora que a máquina do tempo está funcionando no computador dele, vamos enviar isso para o GitHub.
- Ensinamos o `git push`.
- 🔥 **Bônus Matador (Opcional):** Ensinar ele a ativar o "GitHub Pages" lá nas configurações do repositório, para aquele `index.html` virar um site real com um link que ele pode mandar para a mãe dele ver. Isso gera uma pequena vitória (Quick Win) gigantesca!

Capítulo 5: O Próximo Nível (A Oferta do E-book de Python)

- *"Você já sabe controlar versões e já tem um site no ar. Mas para ser contratado, você precisa de um sistema com lógica de verdade no seu GitHub. Que tal criar um Sistema de Cadastro em Python agora?" -> **Link com Desconto.***

Capítulo Bônus: O Cheat Sheet do Programador (Guia de Bolso Git)

"Se der 'branco' na hora de usar o Git, não precisa reler o e-book inteiro. Salve esta página nos seus favoritos ou imprima e cole do lado do monitor. Aqui estão os comandos de sobrevivência que você vai usar todos os dias."

CAPÍTULO 1: O "Hello World" (O Seu Primeiro Cofre de Código)

A maioria dos cursos comete um erro fatal: tentar te ensinar Git usando códigos complexos cheios de arquivos. Isso só gera confusão.

Aqui nós vamos fazer diferente. Nós vamos focar 100% no Git. Por isso, para o nosso laboratório, vamos usar apenas um arquivo HTML super simples. Não se preocupe se você não sabe HTML, você só precisa saber copiar e colar.

PARADA OBRIGATÓRIA: O Seu Ambiente Está Pronto?

Se você já tem o Git instalado no seu computador e a sua conta no GitHub criada, excelente. Pode pular este aviso e ir direto para a prática.

*Mas, se você **não faz ideia de como instalar o Git**, não sabe como criar a conta no GitHub ou configurar o seu terminal, não tente adivinhar.*

*No nosso **Guia Prático: Sistema de Cadastro em Python**, além de ensinar a programar do zero, nós temos um capítulo focado exclusivamente em preparar o seu ambiente de desenvolvimento profissional. Lá, eu te mostro o passo a passo exato para instalar o Python, o VS Code, o Git e vincular a sua conta no GitHub sem erros.*

 [Clique aqui para garantir o E-book de Python e estruturar seu ambiente do zero](#)

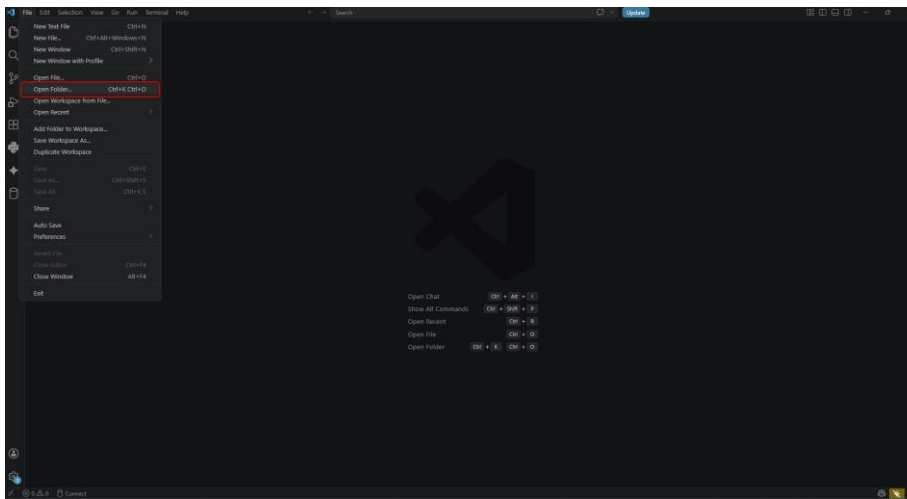
Se o seu PC já está configurado e pronto para a guerra, respire fundo e vamos dominar os comandos.

A partir de agora, pare de imaginar o Git como um monstro de tela preta e comece a imaginá-lo como um **Cofre de Alta Tecnologia** ou uma **Máquina do Tempo**.

Siga o passo a passo comigo:

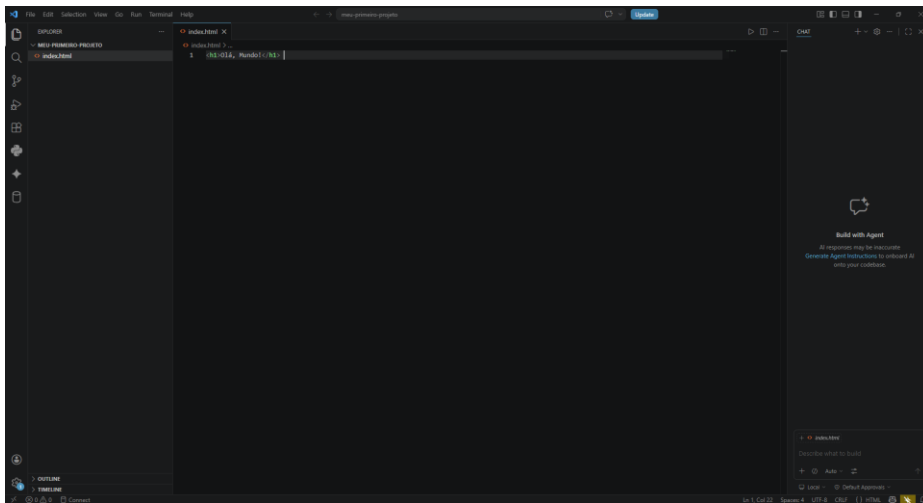
Passo 1: Preparando o Terreno

1. Crie uma pasta nova no seu computador (na sua Área de Trabalho mesmo) e chame de meu-primeiro-projeto.
2. Abra essa pasta no seu editor de código (como o VS Code).



3. Crie um arquivo chamado `index.html` e digite exatamente isso dentro dele:

`<h1>Olá, Mundo!</h1>`



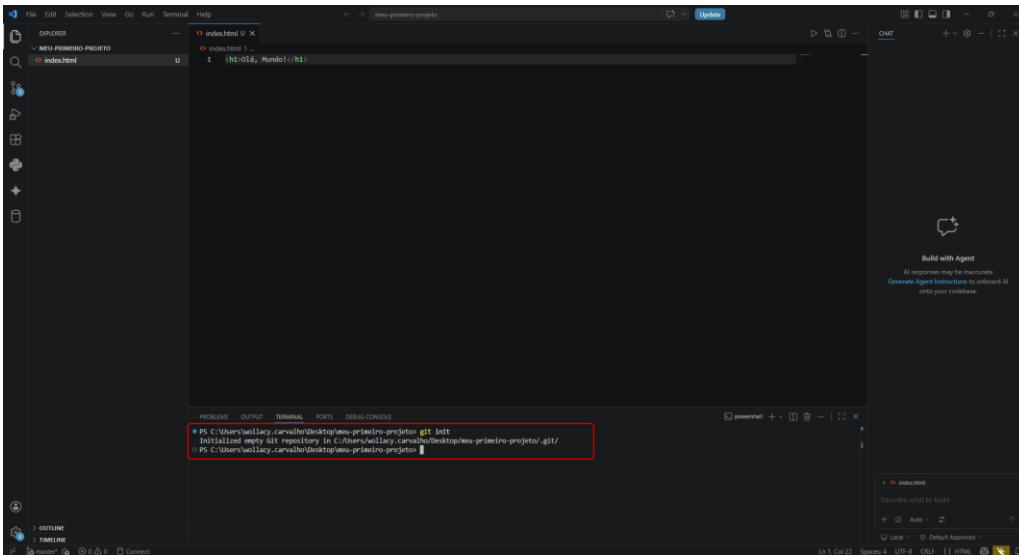
4. Salve o arquivo.

Pronto. Você tem um projeto. Mas, por enquanto, essa pasta é só uma pasta normal e burra. Se você apagar esse arquivo sem querer, ele foi para o espaço. Vamos mudar isso agora.

Passo 2: O Despertar do Git (`git init`)

Abra o terminal do seu editor de código (no VS Code, vá em *Terminal* > *New Terminal*). Agora, você vai dar o seu primeiro comando mágico. Digite:

👉 `git init` e aperte Enter.



O que acabou de acontecer? Você acabou de transformar a sua pasta comum em um cofre monitorado pelo Git. A partir deste exato milissegundo, o Git está escondido na sua pasta, de olhos bem abertos, vigiando cada vírgula que você altera, cria ou deleta.

Passo 3: O Palco de Fotos (`git add`)

O Git está vigiando, mas ele não salva as coisas automaticamente. Você precisa dizer a ele o que é importante. Imagine que você vai tirar uma foto oficial da sua equipe. Antes de bater a foto, você precisa chamar as pessoas e colocá-las no palco, certo? O comando `add` faz exatamente isso.

No terminal, digite: 👉 `git add index.html` (ou apenas `git add .` para adicionar tudo de uma vez) e aperte Enter.

O Git não vai te responder nada (ele é silencioso quando dá certo). Mas o que aconteceu nos bastidores é que você pegou o seu arquivo `index.html` e colocou ele no "palco", pronto para a foto.

Passo 4: Batendo a Foto Oficial (`git commit`)

Agora que o arquivo está no palco, nós vamos finalmente registrar essa versão na nossa máquina do tempo. Vamos "bater a foto" e colocar uma etiqueta nela para a gente lembrar no futuro.

Digite no terminal: 👉 `git commit -m "Meu primeiro commit"` e aperte Enter.

Parabéns! Você acabou de criar a sua primeira versão oficial. A letra `-m` significa *Message* (Mensagem). O texto entre aspas é a etiqueta da sua foto. Se o seu computador explodisse agora (e você já tivesse mandado isso para a nuvem, o que vamos aprender em breve), esse exato arquivo com o "Olá, Mundo!" estaria salvo para sempre.

Parece bobo, mas você acabou de executar o exato fluxo de trabalho que um Engenheiro de Software Sênior do Google faz todos os dias: `init`, `add` e `commit`.

No próximo capítulo, nós vamos evoluir esse nosso código e ver como o Git cria o histórico de versões para você nunca mais perder uma linha de trabalho.

CAPÍTULO 2: A Evolução (Criando o Seu Histórico)

No capítulo anterior, nós colocamos o seu código dentro do cofre e batemos a primeira foto oficial.

Mas o mundo real não é estático. Um software está sempre evoluindo. Você vai adicionar novos botões, mudar textos, corrigir bugs. E é exatamente aqui que o Git brilha: ele cria uma linha do tempo perfeita de cada vírgula que você altera.

Vamos evoluir o nosso projeto e ver como o Git reage a isso.

Passo 1: Modificando o Projeto

Abra o seu arquivo `index.html` lá no VS Code. Logo abaixo do `<h1>`, adicione um parágrafo novo. O seu código deve ficar exatamente assim:

```
<h1>Olá, Mundo!</h1>
<p>Estou aprendendo Git na prática!</p>
```

Salve o arquivo (Ctrl + S ou Cmd + S).

Passo 2: O Radar do Git (`git status`)

Lembra que eu disse que o Git fica escondido na sua pasta vigiando tudo? Vamos testar se isso é verdade. No seu terminal, digite o seguinte comando:

👉 `git status` e aperte Enter.

O Git vai te responder com algumas linhas de texto, e você vai ver o nome `index.html` escrito em **vermelho**. O comando `status` é o seu radar. Sempre que você estiver perdido, digite `git status`. O que o Git está te dizendo com esse texto em vermelho é: *"Opa! Eu notei que você modificou o arquivo `index.html` desde a nossa última foto oficial, mas você ainda não colocou essa alteração no palco."*

Passo 3: Repetindo o Ciclo de Sucesso

Você já sabe o que fazer. Precisamos colocar a alteração no palco e bater uma nova foto.

1. Digite 👉 `git add .` (para colocar no palco).
2. Se você der um `git status` agora, vai ver que o texto ficou **verde**. O Git está dizendo: *"Beleza, a alteração está no palco, pronta para a foto!"*
3. Digite 👉 `git commit -m "Adicionando um paragrafo"` e aperte Enter.

Pronto! Segunda foto oficial registrada com sucesso.

Passo 4: O Painel da Máquina do Tempo (`git log`)

Agora vem a parte onde a sua mente vai explodir. Você bateu duas fotos. Onde elas ficam guardadas? Como você pode ver o histórico do seu trabalho?

No terminal, digite o comando mágico: 🖱️ `git log` e aperte Enter.

Olhe bem para a tela do seu terminal. O Git acabou de te listar de forma incrivelmente detalhada o histórico do seu projeto. Você vai ver dois blocos de texto (seus dois commits). Em cada um deles, o Git te mostra:

- **O Autor:** Quem fez a alteração (seu nome e e-mail).
- **A Data:** O dia, mês, ano e o segundo exato em que a alteração foi salva.
- **A Mensagem:** A etiqueta que você colocou ("Meu primeiro commit" e "Adicionando um paragrafo").
- **O Hash:** Uma sequência gigante de letras e números amarelos. Essa é a "impressão digital" única da sua versão.

Neste exato momento, você tem um histórico inviolável do seu projeto.

Mas e se amanhã você acordar de mau humor, apagar metade do seu código sem querer e salvar? Como usamos esse histórico para voltar no tempo e desfazer uma cagada monumental?

É exatamente o que vamos descobrir no próximo capítulo com o nosso "Botão de Pânico".

CAPÍTULO 3: O Botão de Pânico (A Máquina do Tempo)

Todo programador iniciante tem um medo em comum: o medo de mexer no que está quieto, apagar um código que estava funcionando perfeitamente e destruir o projeto inteiro.

Se você não usa Git, o seu único "salva-vidas" é rezar para o Ctrl + Z funcionar. Mas e se você já fechou o arquivo? E se o computador reiniciou? O Ctrl + Z some e o seu trabalho vai para o lixo.

Como nós temos o nosso cofre blindado configurado, o Ctrl + Z é coisa do passado. Nós temos uma **Máquina do Tempo**. Vamos testar isso na prática simulando um desastre.

Passo 1: O Desastre Planejado

Abra o seu arquivo `index.html` no VS Code. Agora, selecione absolutamente todo o código que está lá e **APAGUE**. Deixe a tela em branco. Salve o arquivo (Ctrl + S). Pode até fechar o arquivo e abrir de novo, só para ter certeza de que o Ctrl + Z não vai te salvar.

Pronto. Você acabou de destruir o seu projeto e perder o seu trabalho. Se bater o desespero e você digitar `git status` no terminal, o Git vai mostrar o arquivo em vermelho, te avisando que ele foi modificado (neste caso, esvaziado).

Mas calma. Não precisa entrar em pânico.

Passo 2: O Resgate (`git restore`)

Como nós não colocamos esse erro no palco (`git add`) e não batemos a foto oficial (`git commit`), o Git sabe perfeitamente que essa alteração não foi aprovada por você.

A última foto oficial que o Git tem guardada é aquela com o código perfeito do Capítulo 2. Nós só precisamos pedir para ele buscar aquela versão.

No seu terminal, digite o comando de resgate: 🖱️ `git restore index.html` e aperte Enter.

Passo 3: A Mágica

Volte para o seu arquivo `index.html` (se ele estiver aberto no VS Code, você vai ver a mágica acontecer ao vivo).

Todo o seu código voltou. Intacto. Exatamente como estava no momento do seu último commit.

Você acabou de viajar no tempo! O comando `restore` simplesmente diz ao Git: *"Esquece tudo o que eu fiz de errado agora e puxa a última versão oficial segura que você tem no cofre"*.

É por isso que os desenvolvedores profissionais fazem *commits* o tempo todo. Cada vez que eles terminam uma funcionalidade que dá certo, eles "batem a foto". Se a próxima funcionalidade der

errado e quebrar tudo, eles apertam o botão de pânico (`git restore`), voltam para a versão segura de 5 minutos atrás e tentam de novo, sem perder noites de sono.

Agora você está 100% protegido no seu próprio computador. O próximo e último passo definitivo é pegar esse cofre perfeito que está na sua máquina e mandá-lo para as nuvens, para que os recrutadores possam ver o seu talento.

Bem-vindo ao GitHub. É o que veremos no próximo capítulo.

CAPÍTULO 4: Subindo para as Nuvens (O Seu Portfólio no GitHub)

Até agora, o seu cofre e a sua máquina do tempo estão funcionando perfeitamente, mas eles estão presos dentro do seu computador.

Se o seu HD queimar hoje, você perde tudo. E o mais importante: se um recrutador quiser ver como você escreve código, ele não tem como acessar a sua máquina.

É para isso que o **GitHub** existe. Ele é como se fosse o "Instagram dos Programadores". É a rede social onde você expõe o seu trabalho para o mundo e, principalmente, para as empresas que estão contratando.

Vamos pegar aquele nosso projeto `meu-primeiro-projeto` e jogá-lo nas nuvens.

Passo 1: Criando o Cofre Virtual

1. Acesse o seu perfil no [GitHub.com](https://github.com).
2. No canto superior direito, clique no botão verde **"New"** (Novo repositório).
3. No nome do repositório, digite: `meu-primeiro-projeto`.
4. Role a página até o final e clique no botão verde **"Create repository"**.

O GitHub vai te mostrar uma tela cheia de códigos confusos. Ignore quase tudo. Nós só precisamos conectar o seu computador a esse cofre virtual que você acabou de criar.

Passo 2: O Elo de Ligação (`git remote`)

Volte para o terminal do seu VS Code (onde o seu projeto HTML está aberto). Nós precisamos dizer ao seu Git local para onde ele deve enviar as fotos oficiais.

Lá na tela do GitHub que você acabou de criar, procure por uma linha parecida com essa (mas com o seu nome de usuário) e copie: `git remote add origin https://github.com/SEU-USUARIO/meu-primeiro-projeto.git`

Cole isso no seu terminal e aperte Enter. Mais uma vez, o Git será silencioso. Mas o que ele fez foi criar uma ponte invisível entre o seu computador e os servidores do GitHub.

Passo 3: O Lançamento (`git push`)

Agora é a hora do show. Vamos pegar todo aquele nosso histórico de commits (as fotos oficiais) e empurrar pela ponte até chegar na nuvem.

No terminal, digite: `👉 git push -u origin main` (ou `master`, dependendo de como seu Git está configurado) e aperte Enter.

Você vai ver algumas linhas de carregamento. Quando terminar, volte para a página do GitHub no seu navegador e **atualize a página** (F5).

BOOM! 🌟 O seu arquivo `index.html`, com o código exato que você escreveu, está lá. Salvo na nuvem, com o histórico de commits intacto e pronto para ser acessado de qualquer lugar do planeta.

Você acabou de dominar a base da ferramenta mais importante do mundo da programação.

CAPÍTULO 5: O Maior Erro dos Iniciantes (E o Seu Próximo Passo)

Parabéns! Se você chegou até aqui e executou os passos, você já sabe controlar as versões do seu código, desfazer erros graves e enviar tudo para o GitHub.

Você acabou de dominar a base da ferramenta mais importante do mundo da programação. Você já está na frente de 80% das pessoas que tentam aprender sozinhas e travam nessa exata etapa.

Mas eu preciso ser 100% sincero com você. Agora que você tem o poder do GitHub nas mãos, você corre o risco de cometer o **maior erro de quem está começando**.

A Síndrome do GitHub Vazio

Sabe o que faz um recrutador olhar o seu perfil e decidir te chamar para uma entrevista? **O que tem dentro dele.**

Nenhuma empresa vai te contratar por ter um arquivo `index.html` com um "Hello World" escrito. O mercado procura resolvedores de problemas. Eles querem ver **lógica, sistemas reais e códigos em linguagens de peso, como o Python**.

Se o seu GitHub está vazio, ou cheio de códigos copiados de tutoriais genéricos, ele é apenas uma vitrine sem produtos. Você tem a loja, mas não tem o que vender.

O Seu Próximo Degrau

Você já deu o primeiro passo e provou que é alguém que executa. Agora, o seu objetivo número um deve ser inaugurar o seu GitHub com um **projeto profissional de verdade**.

Foi exatamente para preencher essa lacuna que eu criei o **Guia Prático: Sistema de Cadastro em Python**.

Nele, eu pego você pela mão e te ensino a construir um sistema real do zero absoluto. Sem vídeo-aulas enroladas de 40 minutos, sem teoria chata que você esquece no dia seguinte. É você, o código e o resultado na tela, do mesmo jeito prático que fizemos neste laboratório de Git.

O que você vai dominar:

- A base da linguagem Python (a mais requisitada pelo mercado hoje).
- Como estruturar a lógica de um sistema do zero.
- Como receber, processar e guardar dados de clientes.
- E o mais importante: ao final da leitura, você terá um **Sistema de Cadastro completo** funcionando na sua máquina, pronto para você fazer o seu primeiro grande `git push` e colocar no seu portfólio.

O Seu Mapa de Evolução (Escolha o seu próximo passo)

Como você baixou este Guia de Git e chegou até o final, você provou ser um executor. E para executores, eu não ofereço apenas um caminho, eu ofereço um **ecossistema completo** para você evoluir, não importa em qual nível você esteja hoje.

Eu liberei um **Desconto exclusivo** em todos os meus materiais oficiais. Escolha o seu próximo desafio abaixo, clique no link e destrave o seu acesso imediato:

🟢 **NÍVEL 1: Do Zero ao Primeiro Sistema** Se você quer aprender o básico da programação na prática e inaugurar seu GitHub construindo um Sistema de Cadastro em Python:

👉 [[CLIQUE AQUI PARA GARANTIR O E-BOOK 1 \(SISTEMA DE CADASTRO\)](#)]

🟡 **NÍVEL 2: Subindo a Régua** Se você já sabe o básico do Python e quer ir além, dominando conceitos mais complexos para criar aplicações mais robustas:

👉 [[CLIQUE AQUI PARA GARANTIR O E-BOOK 2 \(PYTHON INTERMEDIÁRIO\)](#)]

🟠 **NÍVEL 3: Academia de Lógica** Se você já estudou a teoria, mas "dá branco" na hora de resolver problemas reais, você precisa criar memória muscular. Se quiser treinar intensamente a sua lógica:

👉 [[CLIQUE AQUI PARA GARANTIR O PACK DE EXERCÍCIOS PRÁTICOS](#)]

🟢 **NÍVEL 4: O Coração dos Sistemas** Nenhum sistema profissional sobrevive sem guardar informações de verdade. Se você quer dominar a linguagem universal dos bancos de dados e aprender SQL:

👉 [[CLIQUE AQUI PARA GARANTIR O E-BOOK DE SQL](#)]

A ferramenta de versionamento (Git) você já tem. A prateleira de conhecimentos está aberta. Agora só falta construir a sua obra de arte.

Não deixe o seu GitHub pegando poeira. Escolha o seu próximo degrau, clique no link acima e comece a programar o seu futuro hoje mesmo.

Vejo você do outro lado! 🚀

BÔNUS: O Cheat Sheet do Programador (Guia de Bolso Git)

Se der "branco" na hora de usar o Git, não precisa reler o e-book inteiro. Salve esta página nos seus favoritos ou imprima e cole do lado do monitor. Aqui estão os comandos de sobrevivência que você vai usar todos os dias.

1. Os Comandos do Dia a Dia (O Ciclo Básico)

Esses são os três comandos que você vai usar 90% do tempo. É o arroz com feijão do Git.

- **git init:** Transforma a pasta normal em um cofre monitorado pelo Git. Só precisa rodar uma vez na vida do projeto.
- **git status:** O seu radar. Mostra quais arquivos foram modificados e quais ainda não estão no palco. *Dica: Se estiver perdido, digite isso primeiro.*
- **git add .:** Coloca todos os arquivos modificados no "palco" preparando para a foto. Se quiser adicionar só um arquivo, use `git add nome_do_arquivo`.
- **git commit -m "Sua mensagem aqui":** Bate a foto oficial e salva a versão com uma etiqueta de identificação.

2. A Ponte para a Nuvem (Comandos do GitHub)

Para mandar ou puxar arquivos do seu GitHub para o seu computador.

- **git push origin main:** Envia todas as suas fotos oficiais (commits) para o GitHub. (Obs: às vezes o nome pode ser master em vez de main).
- **git pull origin main:** Puxa as atualizações do GitHub para o seu computador. Essencial se você estiver trabalhando em equipe ou em mais de um computador.
- **git clone URL_DO_PROJETO:** Copia um repositório inteiro que está no GitHub para o seu computador. Perfeito para baixar projetos de outras pessoas para estudar.

3. O Botão de Pânico (Corrigindo Merd@s)

Fez besteira? Apagou o que não devia? Use esses comandos para se salvar.

- **git log:** Abre o histórico detalhado. Mostra todas as fotos oficiais que você já tirou, com data, hora e o código (hash) de cada uma.
- **git restore NOME_DO_ARQUIVO:** O salvador de vidas. Desfaz todas as alterações não salvas de um arquivo e volta para a última versão oficial. O verdadeiro Ctrl + Z turbinado.
- **git commit --amend:** Escreveu a mensagem do último commit errado? Esse comando abre o terminal para você corrigir o texto da etiqueta antes de mandar para a nuvem.

4. Comandos de Ramificação (Para Entrevistas de Emprego)

Quando você for trabalhar em equipe, nunca vai codar direto na versão oficial (main). Você vai criar "galhos" (branches) paralelos. Recrutadores amam ver isso.

- **git branch:** Lista todos os "galhos" que existem no seu projeto. O galho que estiver com um asterisco (*) é onde você está no momento.
- **git checkout -b NOME_DA_NOVA_BRANCH:** Cria um "galho" novo paralelo e já te joga para dentro dele. Ex: `git checkout -b teste-botoes`.
- **git checkout NOME_DA_BRANCH:** Pula de um galho para outro sem criar nada novo.
- **git merge NOME_DA_BRANCH:** Junta o galho que você estava trabalhando com o galho principal, unindo os códigos de forma segura.